HAIMEDA Technical Documentation: Fine-Tuning and System Configuration

Paul Sigloch[1] and Christoph Benzmüller[1,2]

[1] University of Bamberg, Bamberg, Germany
[2] Free University of Berlin, Berlin, Germany
paul.sigloch@uni-bamberg.de, christoph.benzmueller@uni-bamberg.de

This technical documentation complements the main paper, *Neuro-Symbolic Verification of LLM Outputs for Data-Sensitive Domains*, by providing full implementation details for the HAIMEDA (Hybrid AI for Medical Device Assessment) system. Section 1 documents iterative fine-tuning experiments and analyses, while Section 2 reports implementation configuration, dependencies, performance profiling, and usability instrumentation.

1 Fine-Tuning Configuration and Results

Fine-tuning HAIMEDA's language models required three iterative phases, each addressing limitations discovered in the preceding phase. This progression, from an initial exploratory approach through multi-stage optimization to specialized model variants, reflects the empirical nature of large language model (LLM) adaptation for domain-specific applications. The following subsections document configuration parameters, training dynamics, and outcomes for each phase, providing the technical detail necessary for reproducibility while highlighting design decisions that influenced subsequent iterations.

1.1 Phase 1: Initial Fine-Tuning Approach

The first phase served as an exploratory baseline to assess whether a single fine-tuned model could generate usable medical device assessment content. Using 899 historical reports processed into instruction-following format, this phase established data pipelines and identified fundamental challenges that shaped subsequent approaches.

Hyperparameters and Configuration Table 1 summarizes the training configuration for Phase 1. The base model (`Llama3-DiscoLeo-Instruct-8B`) was selected for its multilingual capabilities and compatibility with consumer GPU hardware. Low-Rank Adaptation (LoRA) parameters followed established defaults for instruction tuning.

Table 1: Fine-tuning parameters for Phase 1.

Parameter	**Value**
Training Configuration	
Mode	Supervised
Base Model	Llama3-DiscoLeo-Instruct-8B
Max Sequence Length	3000
Dataset Focus	Mixed instruction data
Use Checkpoint	No
Training Hyperparameters	
Epochs	3
Learning Rate	2×10^{-5}
Batch Size	1×8
Warmup Steps	100
LR Scheduler	Cosine
Max Gradient Norm	1.0
Eval Steps	301
Save Steps	100
Save Total Limit	3
Gradient Checkpointing	Yes
Weight Decay	0.01
LoRA Configuration	
Rank (r)	16
Alpha	32
Dropout	0.05
Target Modules	7 modules

Training Metrics and Loss Curves Tables 2 and 3 present training dynamics and evaluation loss progression. Despite achieving 75.6% loss reduction, practical testing revealed a critical disconnect between quantitative metrics and functional output quality.

Table 2: Training duration, throughput, and learning rate metrics for Phase 1 fine-tuning.

Metric	**Value**
Training Duration	5.75 hours
Samples per Second	0.422
Steps per Second	0.053
Initial Learning Rate	2.0×10^{-6}
Peak Learning Rate	2.0×10^{-5}
Final Learning Rate	2.0×10^{-10}

Note: Learning rate followed a cosine schedule with warmup, peaking at epoch 0.27 and decaying to near-zero by completion.

Table 3: Evaluation loss at key checkpoints during Phase 1 fine-tuning, showing model performance progression across epochs.

Checkpoint	**Epoch**	**Eval Loss**
Pre-Training	0.00	1.6263
Validation 1	0.83	0.4513
Validation 2	1.65	0.3902
Validation 3	2.48	0.3707
Final Test	3.00	0.3962

Note: Metrics collected during fine-tuning of the Llama 3 8B IT model. The decreasing evaluation loss indicates improving model performance through the training process.

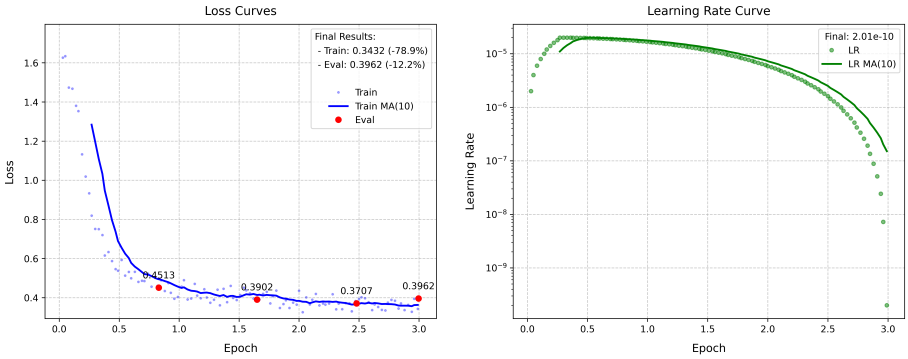

Fig. 1: Training dynamics during Phase 1 fine-tuning: evolution of evaluation loss and learning rate across training epochs.

Outcomes and Limitations Phase 1 revealed a critical disconnect between quantitative metrics and practical performance. Despite achieving 75.6 % loss reduction from 1.626 to 0.396 across three epochs (Table 3), the model failed to generate usable content.

Systematic inference testing exposed a fundamental failure mode: the model consistently generated only chapter titles without corresponding content. Temperature sensitivity analysis quantified this behavior: at $\tau < 0.7$, outputs terminated immediately after titles; within $0.7 \leq \tau \leq 1.0$, full chapters were generated in approximately 15% of cases; and at $\tau > 1.0$, outputs became longer but increasingly incoherent. Root cause analysis identified training data formatting as the primary culprit. Each instruction example contained chapter titles followed by two consecutive newlines before content, creating an artificial prediction boundary. The model optimized for title prediction based on input metadata rather than generating coherent chapter content, a failure mode that standard loss metrics did not capture.

These findings established requirements for Phase 2: redesigned instruction templates eliminating artificial stopping points, explicit output structure delineation, and separation of domain knowledge acquisition from instruction-following training.

1.2 Phase 2: Multi-Stage Fine-Tuning and Optimization

Phase 2 addressed Phase 1's failures through progressive training across five sequential stages. The strategy separated domain knowledge acquisition (Stages A–C, unsupervised) from instruction-following capability (Stages D–E, supervised), with each stage building upon the merged adapter from preceding stages.

Hyperparameters and Configuration Table 4 details the progressive parameter modifications across stages. Key variations include increasing sequence lengths (1 100 to 3 200 tokens), evolving LoRA dimensions ($r = 16$ to $r = 24$), and transitioning from linear to cosine-with-restarts learning rate schedules.

Table 4: Fine-tuning parameters for Phase 2. Bold values indicate stage-specific changes or optimizations.

Parameter	**Stage A**	**Stage B**	**Stage C**	**Stage D**	**Stage E**
Training Configuration					
Mode	Unsupervised	Unsupervised	Unsupervised	**Supervised**	**Supervised**
Base Model	Llama3-DiscoLeo-8B	Stage A	Stage B	Stage C	Stage C
Max Sequence Length	1 100	**3 000**	3 100	3 200	3 200
Dataset Focus	MDB texts	Single chapters	Multiple chapters	Chapters & summaries	Chapters
Use Checkpoint	No	**Yes**	No	No	**Yes**
Training Hyperparameters					
Epochs	3	3	4	4	5
Learning Rate	2×10^{-5}	2×10^{-5}	2×10^{-5}	$\mathbf{1.5 \times 10^{-5}}$	$\mathbf{1.5 \times 10^{-5}}$
Batch Size	1×8	1×8	1×8	1×8	1×8
Warmup Ratio	0.03	0.03	0.03	**0.05**	**0.08**
LR Scheduler	Cosine	Cosine	Cosine	Cosine	**Cosine w/restarts**
Max Gradient Norm	0.3	0.3	0.3	0.3	0.3
Eval Steps	200	350	45	210	210
Save Steps	100	200	40	200	200
Save Total Limit	—	—	—	3	5
Gradient Checkpointing	Yes	Yes	Yes	Yes	Yes
LoRA Configuration					
Rank (r)	16	16	16	**24**	**24**
Alpha	32	32	32	**48**	**48**
Dropout	0.10	0.10	**0.15**	0.10	0.10
Target Modules	All (10)	**Reduced (8)**	All (10)	All (10)	All (10)

Training Metrics and Loss Curves Tables 5 and 6 as well as Figures 2 and 3 document training progression across all five stages. The supervised stages (D–E) achieved substantially better final loss values than unsupervised stages, with Stage E reaching 0.322 after extended training with cosine restarts.

Table 5: Training duration, throughput, and learning rate metrics for each fine-tuning stage (A–E) in Phase 2.

Metric	**Stage A**	**Stage B**	**Stage C**	**Stage D**	**Stage E**
Training Duration	2.05 hours	11.45 hours	1.13 hours	6.38 hours	12.9 hours
Samples per Second	0.921	2.727	0.392	1.769	0.893
Steps per Second	0.115	0.341	0.048	0.048	0.112
Initial Learning Rate	2.0×10^{-6}	2.0×10^{-6}	2.0×10^{-6}	1.5×10^{-6}	1.2×10^{-6}
Peak Learning Rate	2.0×10^{-5}	2.0×10^{-5}	2.0×10^{-5}	1.5×10^{-5}	1.5×10^{-5}
Final Learning Rate	4.5×10^{-9}	5.37×10^{-6}	1.9×10^{-7}	1.33×10^{-5}	3.1×10^{-6}

Note: Stages A–D used cosine learning rate schedules, while Stage E employed a cosine schedule with restarts.

Table 6: Evaluation metrics during fine-tuning Phase 2, showing loss values at key checkpoints for each training stage (A–E).

Checkpoint	**Stage A**		**Stage B**		**Stage C**		**Stage D**		**Stage E**	
	Epoch	**Loss**	**Epoch**	**Loss**	**Epoch**	**Loss**	**Epoch**	**Loss**	**Epoch**	**Loss**
Pre-Training	0.00	2.4333	0.00	2.0385	0.00	0.6657	0.00	1.3087	1.00	1.2581
Validation 1	0.72	0.8018	0.35	0.9032	0.90	0.7292	0.20	0.5698	1.01	0.4521
Validation 2	1.43	0.7261	0.69	0.8257	1.81	0.7204	0.39	0.5297	1.21	0.4340
Validation 3	2.15	0.6949	1.04	0.7961	2.71	0.7164	0.59	0.5087	1.42	0.4223
Validation 4	2.86	0.6873	1.39	0.7730	3.62	0.7142	0.79	0.4922	1.62	0.4122
Validation 5	–	–	1.74	0.7594	–	–	–	–	1.82	0.3846
Validation 6	–	–	2.08	0.7525	–	–	–	–	2.02	0.3908
Validation 7	–	–	2.43	0.7450	–	–	–	–	2.23	0.3764
Validation 8	–	–	2.78	0.7432	–	–	–	–	2.43	0.3689
Validation 9	–	–	–	–	–	–	–	–	2.63	0.3622
Validation 10	–	–	–	–	–	–	–	–	2.83	0.3564
Validation 11	–	–	–	–	–	–	–	–	3.04	0.3528
Validation 12	–	–	–	–	–	–	–	–	3.24	0.3475
Validation 13	–	–	–	–	–	–	–	–	3.44	0.3440
Validation 14	–	–	–	–	–	–	–	–	3.65	0.3414
Validation 15	–	–	–	–	–	–	–	–	3.85	0.3392
Validation 16	–	–	–	–	–	–	–	–	4.05	0.3376
Validation 17	–	–	–	–	–	–	–	–	4.25	0.3368
Validation 18	–	–	–	–	–	–	–	–	4.45	0.3363
Validation 19	–	–	–	–	–	–	–	–	4.66	0.3360
Validation 20	–	–	–	–	–	–	–	–	4.86	0.3360
Final Test	3.00	0.6880	3.00	0.7653	3.94	0.6389	1.00	0.4779	5.00	0.3224

Note: Stages A–C (unsupervised training) used progressively more complex corpus data, while Stages D–E (supervised training) demonstrated significant improvements with instruction fine-tuning.

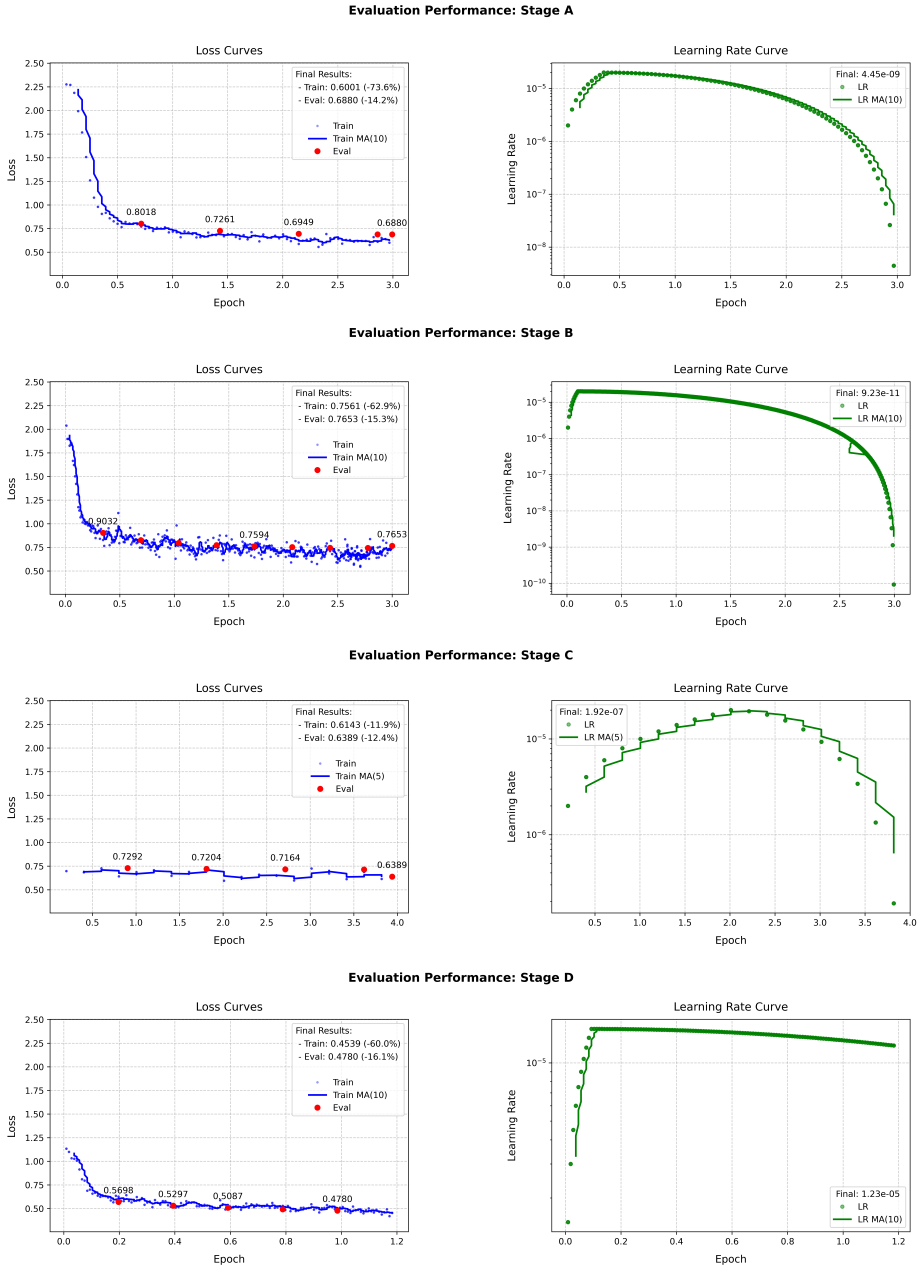

Fig. 2: Training dynamics during Phase 2 fine-tuning: evolution of evaluation loss and learning rate across training epochs for Stages A through D.

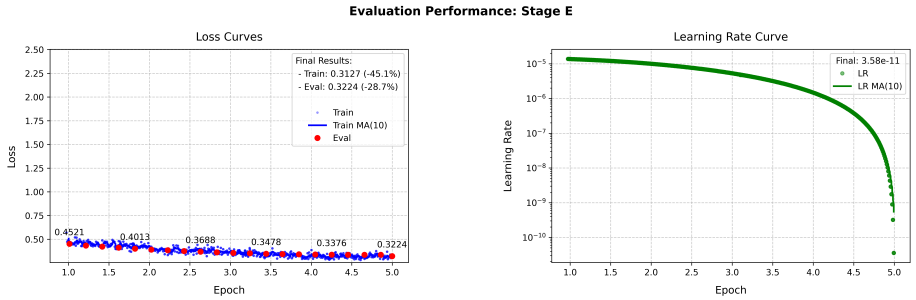

Fig. 3: Training dynamics during Phase 2 fine-tuning: evolution of evaluation loss and learning rate across training epochs for Stage E.

Outcomes and Limitations Phase 2 addressed Phase 1's failures through five sequential stages: three unsupervised (A–C) for domain knowledge acquisition, followed by two supervised (D–E) for instruction-following capability.

The training progression yielded distinct outcomes per stage. Stage A established foundational knowledge using extracted Microsoft Access database (MDB) texts (71.7 % loss reduction). Stage B extended this with single chapters (62.5 % reduction). Stage C implemented multiple chapter contexts but yielded minimal improvement (4.0 %), indicating diminishing returns for unsupervised learning after domain knowledge saturation. The supervised stages achieved more consistent improvement, with Stage E reaching the best final loss of 0.322 (74.4 % reduction) across 20 validation checkpoints.

Testing confirmed functional chapter generation: the model reliably produced complete content rather than titles alone. However, structural coherence remained inconsistent across chapter types. Systematic analysis identified several contributing factors: catastrophic forgetting during sequential training caused gradual loss of document structure understanding while optimizing for domain content [3]; format inconsistencies from MDB table conversion confused document organization [1]; and competing optimization goals created tension between content accuracy and structural consistency [4].

Stage C's minimal improvement validated a key insight: supervised fine-tuning offers more efficient improvement pathways than continued unsupervised training once domain knowledge is established. The observation that sequential training created competing optimization goals motivated Phase 3's architectural change to parallel specialized model variants.

1.3 Phase 3: Specialized Model Training and Merging

Phase 3 departed from sequential training by developing three model variants in parallel: V1 (domain knowledge), V2 (document structure), and V3 (merged via SLERP with coherence fine-tuning). An additional variant, V1-B, specialized V1 for question-answering tasks in the Research and Investigation Module.

Hyperparameters and Configuration Table 7 presents configurations for all Phase 3 variants. Notable techniques include partial layer freezing for V3's coherence fine-tuning (unfreezing layers 8–14 and 20–25 while preserving a frozen bridge at layers 15–19) and specialized learning rate schedules tailored to each variant's objectives.

Table 7: Fine-tuning parameters for Phase 3.

Parameter	V1		V2		V3 (Coherence)	
	Stage A	**Stage B**	**Stage A**	**Stage B**	**Stage A**	**Stage B**
Training Configuration						
Mode	Unsupervised	Supervised	Supervised	Supervised	Unsupervised	Supervised
Base Model	Llama3-DiscoLeo-8B	V1-A	Llama3-DiscoLeo-8B	V2-A	Merged V1 +V2 Model	V3-A
Dataset Focus	Mixed chapters	Questions & answers	Full chapters	Chapters & Summaries	Unsupervised coherence	Supervised coherence
Max Sequence Length	2 900	3 200	3 200	3 200	2 900	3 100
Model Freezing Strategy						
Freeze Strategy	Partial freeze	Partial freeze	Partial freeze	Partial freeze	Selective unfreeze	Selective unfreeze
Frozen Layers	22 of 32	20 of 32	16 of 32	20 of 32	All except 8–14, 20–25	All except 8–14, 20–25
Training Hyperparameters						
Epochs	7	5	4	3	1	1
Learning Rate	2.5×10^{-5}	2.2×10^{-5}	1.2×10^{-5}	1.0×10^{-5}	5.0×10^{-6}	5.0×10^{-6}
Gradient Acc. Steps	6	8	12	12	8	8
Effective Batch Size	6	8	12	12	8	8
Warmup Ratio	0.06	0.06	0.08	0.08	0.15	0.15
LR Scheduler	Linear	Linear	Cosine w/restarts	Cosine w/restarts	Constant w/warmup	Constant w/warmup
Weight Decay	0.01	0.01	0.01	0.02	0.005	0.005
LoRA Configuration						
Rank (r)	32	32	32	32	8	8
Alpha	64	64	72	72	16	16
Dropout	0.10	0.10	0.08	0.12	0.05	0.05
Target Modules	All (10)	All (10)	All (10)	All (10)	All (10)	All (10)

Training Metrics and Loss Curves Tables 8 and 9 as well as Figures 4 and 5 document training dynamics for each variant. V1-B achieved the highest loss reduction (86.9%) during Q&A fine-tuning, while V3's coherence training showed more modest quantitative improvement (38.1%) despite superior qualitative performance.

Table 8: Training duration, throughput, and learning rate metrics for each model variant (V1–V3) and stage (A, B) in Phase 3 fine-tuning.

Metric	**V1-A**	**V1-B**	**V2-A**	**V2-B**	**V3-A**	**V3-B**
Model	Unsupervised	Supervised	Supervised	Supervised	Merged Model	Supervised
Training Duration	45.93 hours	46.62 hours	27.26 hours	18.75 hours	1.27 hours	1.86 hours
Samples per Second	0.397	0.341	0.348	0.347	0.359	0.314
Steps per Second	0.066	0.043	0.029	0.029	0.045	0.039
Initial Learning Rate	2.5×10^{-6}	2.2×10^{-6}	1.2×10^{-6}	1.0×10^{-6}	5.0×10^{-7}	5.0×10^{-7}
Peak Learning Rate	2.5×10^{-5}	2.2×10^{-5}	1.2×10^{-5}	1.0×10^{-5}	5.0×10^{-6}	5.0×10^{-6}
Final Learning Rate	1.79×10^{-5}	1.33×10^{-5}	3.94×10^{-10}	7.21×10^{-10}	5.0×10^{-6}	5.0×10^{-6}

Table 9: Evaluation metrics during fine-tuning Phase 3, showing loss values at key checkpoints for each model variant (V1–V3) and stage (A, B).

Checkpoint	V1				V2				V3			
	Stage A		Stage B		Stage A		Stage B		Stage A		Stage B	
	Epoch	**Loss**	**Epoch**	**Loss**	**Epoch**	**Loss**	**Epoch**	**Loss**	**Epoch**	**Loss**	**Epoch**	**Loss**
Pre-Training	0.00	1.8649	0.00	1.4673	0.00	1.7858	0.00	0.5788	0.00	0.7485	0.00	0.7260
Validation 1	0.32	0.8565	0.21	0.2565	0.25	0.6997	0.27	0.4437	0.24	0.7262	0.19	0.6742
Validation 2	0.64	0.7925	0.42	0.2435	0.49	0.6105	0.54	0.4156	0.49	0.7032	0.38	0.5361
Validation 3	0.96	0.7592	0.63	0.2347	0.74	0.5692	0.81	0.3956	0.73	0.6862	0.57	0.4806
Validation 4	1.28	0.7458	0.84	0.2288	0.98	0.5407	1.08	0.3771	0.97	0.6761	0.76	0.4606
Validation 5	1.60	0.7252	1.05	0.2246	1.23	0.5194	1.35	0.3606	–	–	0.95	0.4498
Validation 6	1.92	0.7089	1.26	0.2200	1.48	0.5006	1.62	0.3473	–	–	–	–
Validation 7	2.24	0.7039	1.47	0.2151	1.72	0.4838	1.88	0.3368	–	–	–	–
Validation 8	2.56	0.6929	1.68	0.2104	1.97	0.4692	2.15	0.3304	–	–	–	–
Validation 9	2.88	0.6815	1.89	0.2058	2.22	0.4578	2.42	0.3258	–	–	–	–
Validation 10	3.20	0.6777	2.10	0.2002	2.46	0.4475	2.69	0.3236	–	–	–	–
Validation 11	3.52	0.6663	2.31	0.1963	2.71	0.4389	2.96	0.3232	–	–	–	–
Validation 12	3.84	0.6520	2.52	0.1941	2.95	0.4325	–	–	–	–	–	–
Validation 13	4.16	0.6585	2.73	0.1891	3.20	0.4286	–	–	–	–	–	–
Validation 14	4.48	0.6510	2.94	0.1894	3.45	0.4257	–	–	–	–	–	–
Validation 15	4.80	0.6443	3.15	0.1869	3.69	0.4246	–	–	–	–	–	–
Validation 16	5.12	0.6540	3.36	0.1844	3.94	0.4244	–	–	–	–	–	–
Validation 17	5.44	0.6499	3.56	0.1821	–	–	–	–	–	–	–	–
Validation 18	5.76	0.6426	3.77	0.1799	–	–	–	–	–	–	–	–
Validation 19	6.08	0.6477	3.98	0.1777	–	–	–	–	–	–	–	–
Validation 20	6.40	0.6454	4.19	0.1785	–	–	–	–	–	–	–	–
Validation 21	6.71	0.6444	4.40	0.1792	–	–	–	–	–	–	–	–
Validation 22	–	–	4.61	0.1785	–	–	–	–	–	–	–	–
Validation 23	–	–	4.82	0.1775	–	–	–	–	–	–	–	–
Final Test	7.00	0.6325	5.00	0.1916	4.00	0.4307	3.00	0.3275	1.00	0.6937	1.00	0.4494

Note: V1 stages represent the first model version with unsupervised (Stage A) and supervised (Stage B) training. V2 stages show specialized format training. V3 represents coherence fine-tuning with unsupervised (Stage A) and supervised (Stage B) training.

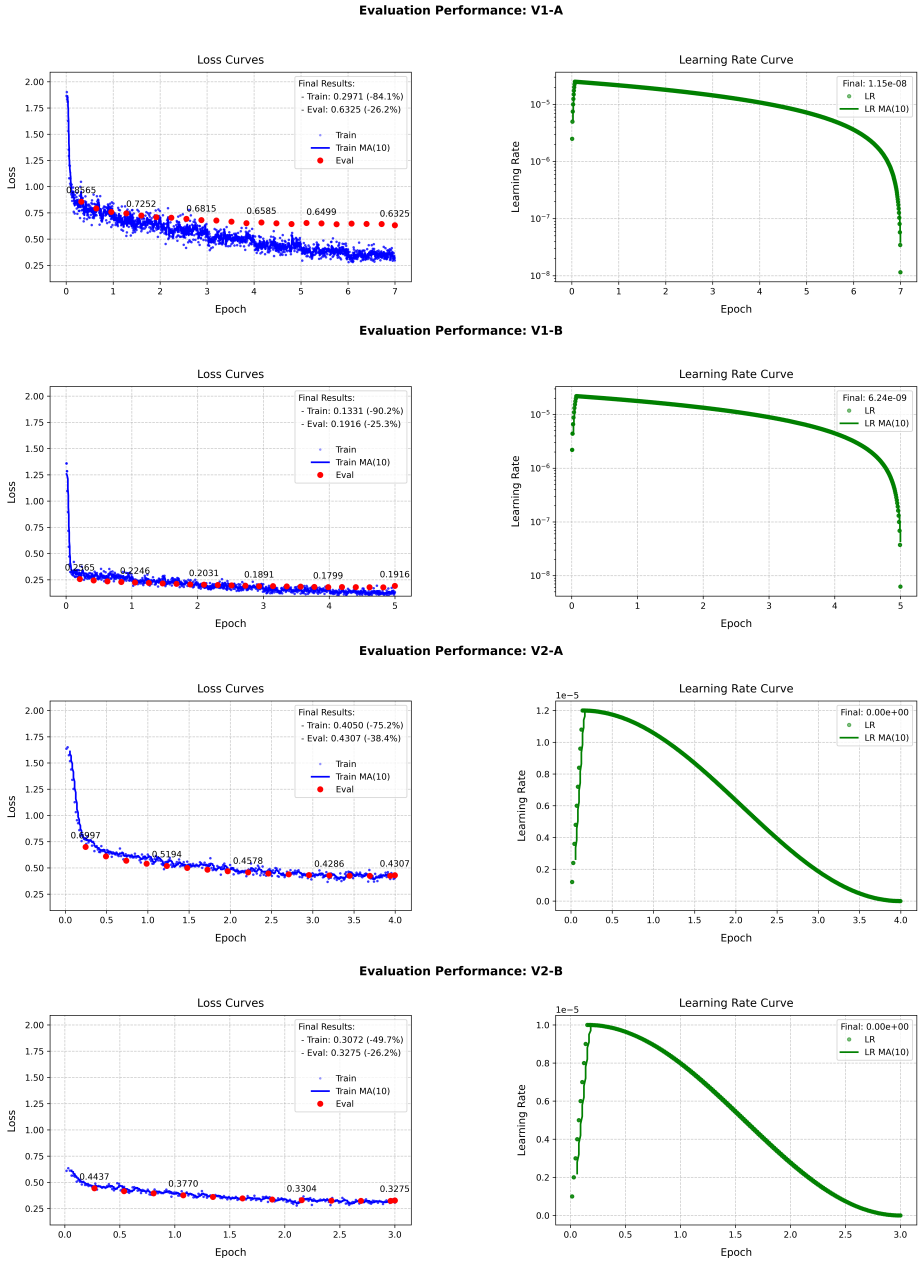

Fig. 4: Training dynamics during Phase 3 fine-tuning: evolution of evaluation loss and learning rate across training epochs for model variants V1 and V2.

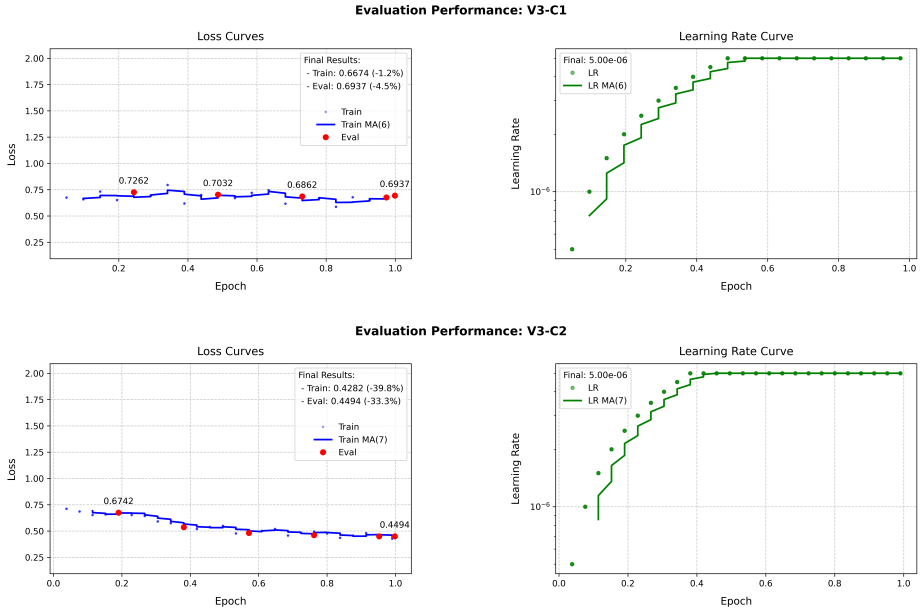

Fig. 5: Training dynamics during Phase 3 fine-tuning: Evolution of evaluation loss and learning rate across training epochs for model variant V3.

Outcomes and Limitations Phase 3 deviated from sequential training by developing three model variants simultaneously: V1 (domain knowledge), V2 (document structure), and V3 (SLERP merged with coherence fine-tuning). An additional variant, V1-B, was created to specialize V1 for question answering.

The training outcomes differed depending on the variant and objective. V1 achieved a 66.1 % loss reduction across seven epochs of unsupervised training. V2 achieved a 75.9 % loss reduction with cosine scheduling and restarts. V1-B achieved the most dramatic reduction (86.9 %) during supervised Q&A fine-tuning, indicating that question-answering represents a more constrained optimization task. V3 demonstrated superior qualitative performance yet showed modest loss reduction (38.1 %) during coherence fine-tuning yet demonstrated superior qualitative performance, illustrating how standard metrics may inadequately capture coherence improvements.

The strategy for partially unfreezing V3 was to target layers 8–14 and 20–25, while preserving a frozen bridge at layers 15–19. This approach was taken to maintain core capabilities from both parent models while enabling coherence-specific adaptation [2].

Testing confirmed improved chapter generation in terms of length and structure. However, two limitations emerged. First, V3 exhibited an increased tendency to hallucinate at low temperatures ($\tau < 0.2$). This involves the introduction of details that are absent from the context. One possible explanation for this phenomenon is that the expanded unfrozen layers enhance generative capability during coherence training, albeit at the expense of factual precision. Second, V1-B performed well with textual queries but showed diminished capability with tabular MDB data, reflecting the training corpus's bias toward textual chapter content [1].

The findings point to the existence of certain trade-offs in the process of specialized fine-tuning. On the one hand, parallel model development with merging improved generation robustness but reduced factual precision. On the other hand, task-specific training improved target performance but reduced cross-format generalization.

2 System Configuration and Evaluation

This section documents implementation-level configuration used during evaluation. Section2.1 summarizes symbolic pattern extraction settings, Section2.2 specifies RIM query decomposition and vector-retrieval configuration, Section2.3 lists core software dependencies, Section2.4 reports measured performance metrics, and Section 2.5 provides usability assessment details.

2.1 Symbolic Pattern Library Configuration

The symbolic entity extraction component uses deterministic pattern matching with extensive regular expression libraries. Table 10 summarizes the pattern coverage for each entity type.

Table 10: Pattern library composition for symbolic entity extraction showing the format coverage for each entity type.

Entity Type	**Pattern Coverage**
Dates	15+ format patterns covering ISO 8601 (YYYY-MM-DD), German conventions (DD.MM.YYYY), English formats (Month DD, YYYY), and locale-specific variations with full and abbreviated month names
Numeric Values	Financial amounts with currency symbols and thousand separators, percentages, quantities with unit specifications, and locale-specific decimal formatting (comma vs. period)
Identifiers	Legal citations (court references, statute numbers), technical codes (device classifications, regulatory identifiers), reference numbers (case IDs, policy numbers), and international standards (ISO, DIN references)
Phrases	Context-aware extraction with configurable placeholder handling for domain-specific terminology

To improve matching robustness across formatting differences, each extracted entity generates multiple representation variants. For instance, a date extracted as "15.01.2024" yields variants such as "15. Januar 2024", "January 15, 2024", and "2024-01-15", which enables successful verification despite formatting inconsistencies between the input and output content. The pattern libraries can be configured to extend and accommodate additional formats or domain-specific patterns without modifying the core verification engine.

2.2 Research and Investigation Module Configuration

Query Decomposition Configuration The Research and Investigation Module (RIM) uses configurable indicator word libraries for symbolic query decomposition. It uses over 40 specialized regular expression patterns for entity-aware query analysis. Query decomposition extracts domain-specific terminology through pattern-based recognition of the following entity categories:

- **Stakeholder References**: Domain-specific party identifiers including insurers, brokers, policyholders, manufacturers, and regulatory bodies
- **Device Categories**: Medical device classifications, product types, and equipment categories
- **Manufacturers**: Company names, brand identifiers, and manufacturer codes
- **Case Identifiers**: Report reference numbers (GA-prefixed identifiers), policy numbers, and claim references
- **Temporal Entities**: Dates, date ranges, and temporal qualifiers
- **Quantitative Entities**: Numeric values, financial amounts, and measurements

Extracted terms are mapped to database schema columns via associations defined in a ruleset and stored in JSON files. A confidence-ranked priority system distinguishes between explicit matches (high priority) and speculative associations (low priority). Multi-table queries execute priority-based filtering and record joining using both exact and fuzzy matching, with configurable similarity thresholds.

RAG and Vector Search Configuration The RIM uses configurable parameters for retrieval-augmented generation (RAG). Table 11 specifies the key configuration values used in the evaluation of HAIMEDA.

Table 11: RAG and vector search configuration parameters.

Parameter	**Value / Description**
Maximum RAG Context	12,000 characters
Chunking Strategy	One chapter per chunk (context-preserving)
Embedding Model	`nomic-embed-text`
Similarity Metric	Cosine distance with character count-based filtering
Vector Collections	
meta_vectors	Extracted metadata from reports (JSON)
report_vectors	Complete report documents (Markdown)
single_chapter_vectors	Individual chapter files

The chunking strategy preserves document context by treating each chapter of a report as a single retrieval unit. This approach respects semantic boundaries rather than imposing fixed token limits. It maintains coherent context for the language model and supports position-aware chunk management that tracks document hierarchies.

Dynamic result sizing adjusts the amount of retrieved content based on the configured context limit. An automatic scaling factor allocates an additional one-third of capacity for structured database results when combining vector search with symbolic query results.

2.3 Software Dependencies

Table 12 lists core software dependencies relevant to reproducibility. The system implements Python integration through ErlPort for computationally intensive ML operations while maintaining Elixir's supervision guarantees for fault tolerance. Statement verification employs a parallel worker pool with adaptive thread allocation based on available GPU memory, enabling efficient batch processing of embedding comparisons.

Table 12: Core software dependencies for HAIMEDA implementation.

Component	**Version**	**Purpose**
Elixir Application Layer		
Phoenix / LiveView	1.7.x / 1.0.x	Web framework and reactive UI
Elixir Desktop	1.5.x	Native Windows deployment
MongoDB Driver	1.0.x	Document database access
ErlPort	0.11.x	Python process integration
LangChain	0.3.x	Prompt template management
Ollama	0.8.x	Local LLM inference API
Python ML Components		
sentence-transformers	2.x	Embedding generation[†]
spaCy	3.x	Keyword extraction (NER, POS)[‡]
scikit-learn	1.x	TF-IDF vectorization

[†]Model: `paraphrase-multilingual-MiniLM-L12-v2`
[‡]Model: `core_news_sm` (language-specific)

2.4 Performance Metrics

Table 13 presents timing and resource utilization measurements across 50 test runs per operation. These metrics characterize computational requirements for the hardware configuration described in the main paper.

Table 13: Detailed performance metrics for HAIMEDA modules and operations, including duration, memory usage, and peak CPU/GPU/VRAM consumption for key system tasks.

Module	**Operation**	**Duration (sec)**	**Process Memory (MB)**	**Peak CPU (%)**	**Peak GPU (%)**	**Peak VRAM (MB)**
System	Initialization	0.90	110.3	23.0	15.0	1,261
IIVM	Pre-processing Input	0.01	100.4	13.4	15.4	19,174
IIVM	Post-processing Output (Symbolic)	0.02	100.7	12.4	15.4	19,150
IIVM	Post-processing Output (Hybrid)	16.58	117.6	26.0	27.0	2,661
RAM	Create Chapter	14.16	102.9	22.6	94.0	19,163
RAM	Revise Chapter Text	6.13	105.2	29.0	94.0	19,190
RAM	Optimize Text	8.75	100.4	30.2	93.8	19,170
RAM	Process User Request (RIM)	19.6	118.5	25.0	93.8	18,350
RIM	Process User Request (RAG)	5.01	116.9	40.0	32.0	18,471
RIM	Process User Request (MDB)	7.91	134.8	30.4	17.0	18,355

Note: Values represent averages across multiple test runs ($n = 50$). System specifications: AMD Ryzen 9 5900X (12 Cores), 64GB RAM (DDR4), Nvidia RTX 4090 with 24GB VRAM.

2.5 Usability Assessment

Table 14 presents System Usability Scale (SUS) responses from the domain expert after integrated workflow testing. The raw score of 62.5 falls within the "marginal" acceptability range, reflecting the system's status as a functional prototype rather than production-ready software.

Table 14: System Usability Scale (SUS) results for HAIMEDA, showing the expert's responses to ten usability statements and the calculated raw SUS score.

Statement	**Strongly Disagree 1**	**Disagree 2**	**Neutral 3**	**Agree 4**	**Strongly Agree 5**
1. I think that I would like to use this system frequently.					**X**
2. I found the system unnecessarily complex.			**X**		
3. I thought the system was easy to use.			**X**		
4. I think that I would need technical support to use this system.			**X**		
5. I found the various functions in this system were well integrated.				**X**	
6. I thought there was too much inconsistency in this system.		**X**			
7. I would imagine that most people would learn to use this system very quickly.			**X**		
8. I found the system very cumbersome to use.		**X**			
9. I felt very confident using the system.				**X**	
10. I needed to learn a lot of things before I could get going with this system.				**X**	

Note: SUS assessment conducted with the surveyor after testing HAIMEDA during regular surveyor activities. Raw SUS score: [62.5] /100.

References

1. Chen, N., Shou, L., Gong, M., Pei, J., You, C., Chang, J., Jiang, D., Li, J.: Bridge the gap between language models and tabular understanding (2023)
2. Jiang, J., Dong, Y., Zhou, J., Zhu, Z.: From compression to expansion: A layerwise analysis of in-context learning (2025)
3. Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., Hadsell, R.: Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy of Sciences **114**(13), 3521–3526 (2017). https://doi.org/10.1073/pnas.1611835114
4. Moukafih, Y., Ghogho, M., Smaili, K.: Supervised contrastive learning as multi-objective optimization for fine-tuning large pre-trained language models. In: ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). pp. 1–5 (2023). https://doi.org/10.1109/ICASSP49357.2023.10095108